

Desenvolvimento de Aplicações Móveis

Prof: Luis Nakamura

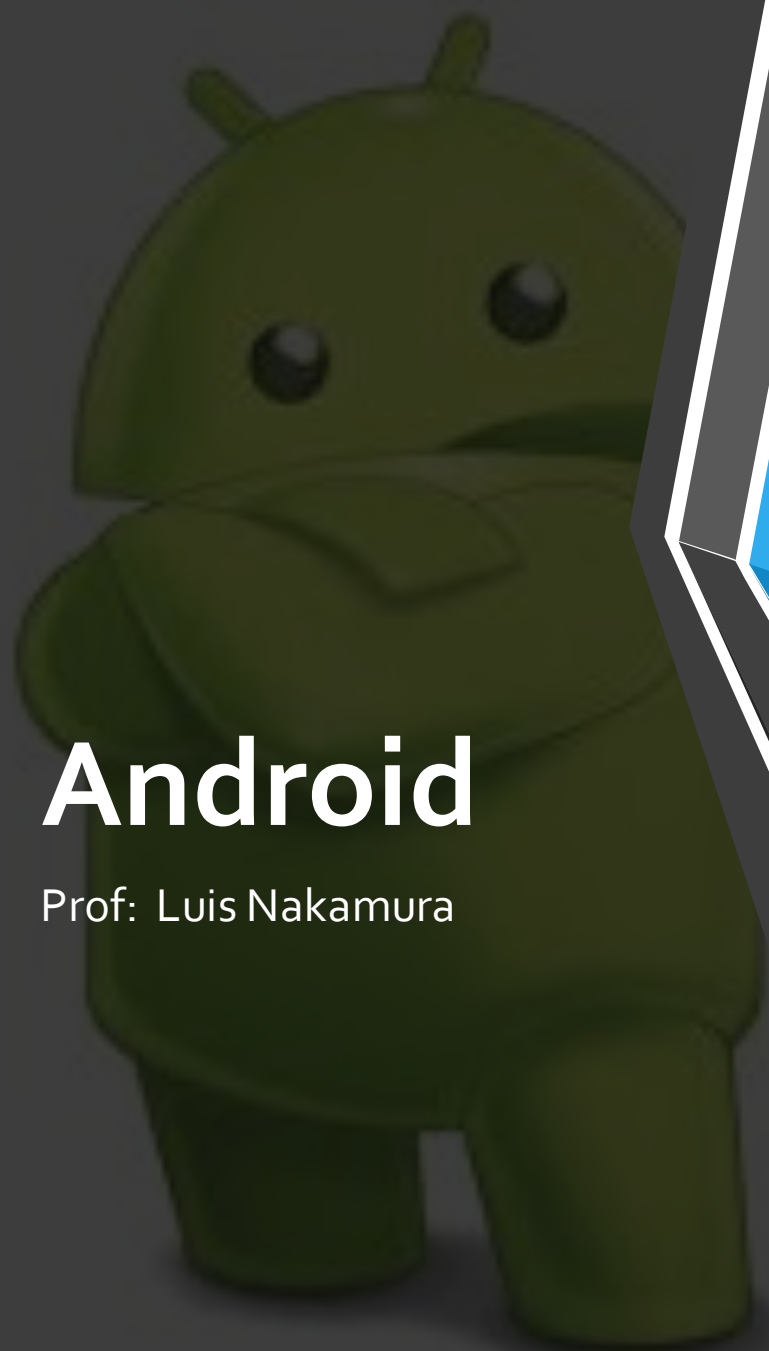


Parte 4 (Continuação)

Apresentação da Aula 4 (Cont.)

- **Objetivos da Aula:**

- Prática: Criando e utilizando o SQLite para armazenamento de dados local.



Android

Prof: Luis Nakamura



Prática: SQLite



- 0) Inserir uma forma (shape)-> Adicionar um novo arquivo xml do tipo shape na pasta drawable: *border.xml*, com o seguinte conteúdo:

```
<shape xmlns:android="http://schemas.android.com/apk/res/android">
    <stroke android:width="2dp" android:color="#FFFFFF" />
    <solid android:color="#000000" />
    <padding android:left="7dp" android:top="7dp"
            android:right="7dp" android:bottom="7dp" />
    <corners android:radius="4dp" />
</shape>
```

Prática: SQLite



- 1) Crie uma Activity com um ListView que exiba o ID e NOME de todas as Pessoas cadastradas no banco de dados (Use o método *SelectAll*);
 - Na activity configurar:
android:background="#000000"
 - Pode-se ajustar as propriedades *padding* (16dp)

```
<ListView
    android:id="@+id/listViewPessoas"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_alignParentLeft="true"
    android:layout_alignParentTop="true"
    android:background="#000000">
</ListView>
```

Prática: SQLite



- 1) Crie uma Activity com um ListView que exiba o ID e NOME de todas as Pessoas cadastradas no banco de dados (Use o método *SelectAll*);
- É preciso adicionar um layout para cada item do listview:

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal"
    android:background="@drawable/border" >
```

Atenção

```
<TextView
    android:id="@+id/textId"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginLeft="5sp"
    android:gravity="center_vertical"
    android:textColor="#FFFFFF" />
```

Continua...

Prática: SQLite



```
<TextView
    android:id="@+id/textSpace"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginLeft="5sp"
    android:gravity="center_vertical"
    android:textColor="#FFFFFF"
    android:text=" - " />
```

```
<TextView
    android:id="@+id/textNome"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginLeft="5sp"
    android:gravity="center_vertical"
    android:textColor="#FFFFFF" />
```

```
</LinearLayout>
```

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
public class ListPessoasAdapter implements ListAdapter {  
  
    //Criamos variaveis que serao utilizadas pelo Adapter  
    //O contexto  
    private Context mContext;  
    //A lista de pessoas  
    private ArrayList<Pessoa> mArray;  
    //Um obj LayoutInflater  
    private LayoutInflater mInflater;
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
//No nosso construtor recebemos o contexto e a lista de pessoas  
public ListPessoasAdapter(Context c, ArrayList<Pessoa> a) {  
  
    //Salvamos o contexto e a lista nas variaveis da classe  
    mContext = c;  
    mArray = a;  
    //e instanciamos nosso layoutinflater utilizando o contexto  
    mInflater = LayoutInflater.from(mContext);  
}
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
@Override
public int getCount() {

    // Retorna o tamanho o ArrayList
    // (quantidade de itens)
    return mArray.size();
}

@Override
public int getViewTypeCount() {

    // Retorna o numero de tipos de Views
    // que o adapter manipula, no nosso caso
    // estamos trabalhando apenas com um tipo
    // de View => TextView
    // IMPORTANTE: Este metodo nao pode
    // zero ou um numero negativo
    return 1;
}
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
@Override  
public Object getItem(int position) {  
  
    //Retornamos um item de acordo com a posicao  
    return mArray.get(position);  
}
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
//Esse e o principal metodo do nosso Adapter
//E nele que retornaremos a View, que no nosso
//caso e um layout (item_listview_pessoa.xml)
//inflado que contem textviews com o id e nome
//da Pessoa
@Override
public View getView(int position, View convertView, ViewGroup parent) {

    //Criamos variaveis locais
    //utilizadas na recuperacao dos
    //componentes
    TextView textViewId;
    TextView textViewNome;
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
//Se a nossa View for nula
//(ainda nao criada ou foi
//reciclada pelo sistema)
if (convertView == null) {

    //devemos recuperar a nossa view,
    //para isso inflamos nosso layout
    convertView =
mInflater.inflate(R.layout.item_listview_pessoa, null);

    //Recuperamos os componentes do nosso layout inflado
    textViewId =
(TextView)convertView.findViewById(R.id.textId);
    textViewNome =
(TextView)convertView.findViewById(R.id.textNome);
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
//Setamos algumas configuracoes iniciais nos componentes
textViewId.setTextColors(Color.parseColor("#FFFFFF"));
textViewNome.setTextColors(Color.parseColor("#FFFFFF"));

} else {

    //caso a view ainda exista
    //apenas recuperas os componentes do nosso layout(view)
    textViewId =
    (TextView)convertView.findViewById(R.id.textId);
    textViewNome =
    (TextView)convertView.findViewById(R.id.textNome);
}
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
//recuperamos o obj Pessoa de acordo com a  
posicao
```

```
Pessoa p = mArray.get(position);
```

```
//configuramos o id da pessoa no textview  
adequado
```

```
textViewId.setText(String.valueOf(p.getId()));
```

```
//configuramos o nome da pessoa no textview  
adequado
```

```
textViewNome.setText(p.getNome());
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
convertView.setOnClickListener(new OnClickListener()
{
    @Override
    public void onClick(View v) {
        Intent viewPessoa = new Intent(mContext,
        PessoaCadastro.class);

        TextView textViewId =
        (TextView)v.findViewById(R.id.textId);

        String pessoaId = textViewId.getText().toString();

        long pessoa_id = Long.parseLong(pessoaId);
```

Continua...

Prática: SQLite



- 1) ...
 - É preciso *Adapter* (classe que implementa a interface *ListAdapter*) para “adaptar” os dados aos os itens do listview:

```
viewPessoa.putExtra(Pessoa.PESSOA_ID, pessoa_id);  
  
viewPessoa.putExtra(Pessoa.PESSOA_READ_ONLY, true);  
  
mContext.startActivity(viewPessoa);  
  
}  
  
});  
  
//o metodo deve retorna nossa view (layout inflado)  
return convertView;  
  
}
```

Continua...

Prática: SQLite



- 1) ...
 - Utilizando o adapter ListPessoasAdapter:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_aula_sqlite);

    showPessoas(this);
}
```

Continua...

Prática: SQLite



- 1) ...
 - Utilizando o adapter ListPessoasAdapter:

```
//Esse metodo exhibe a lista de pessoas na activity  
private void showPessoas(Context context) {  
  
    //Encontramos a nossa listview  
    ListView lvPessoas = (ListView)  
    findViewById(R.id.listViewPessoas);  
  
    //Criamos um obj da nossa classe DBSQLiteAccess para  
acessar o DB  
    DBSQLiteAccess dbAccess = new DBSQLiteAccess(this);  
  
    //Recuperamos a lista de pessoas com o metodo  
selectAll  
    ArrayList<Pessoa> arrPessoas = dbAccess.selectAll();
```

Continua...

Prática: SQLite



- 1) ...
 - **Utilizando o adapter** ListPessoasAdapter:

```
/Setamos um adapter (ListPessoasAdapter) passando para ela a  
nossa  
//lista de pessoas, o adapter ira processar e fazer o  
listview exibir nossas pessoas adequadamente  
lvPessoas.setAdapter(new ListPessoasAdapter(this,  
arrPessoas));  
  
}
```

Continua...

Próximas Aulas

- Web Services
- Google Maps
- Etc.

Referências

- <http://www.androidcentral.com/android-versions>
- <http://www.idc.com/prodserv/smartphone-os-market-share.jsp>
- http://www.cisco.com/c/en/us/solutions/collateral/service-provider/visual-networking-index-vni/white_paper_c11-520862.html
- <http://www.android.com/versions/kit-kat-4-4/>
- <http://www.android.com/versions/lollipop-5-0/>
- <http://developer.android.com/about/dashboards/index.html>
- DEITEL, P. J.; DEITEL, H. M.. Android para programadores: uma abordagem baseada em aplicativos. 1. ed. Porto Alegre: Bookman, 2012. 512p.
- FELKER, D.. Android application development for dummies. 1. ed. John Wiley & Sons, 2012. 456p.
- NUDELMAN, G.. Android design patterns: interaction designs. 1. ed. John Wiley & Sons, 2012. 384p.
- DARWIN, I. F.. Android cookbook. 1. ed. São Paulo: Novatec, 2012. 672p.